

# Athlete Intelligence — Complete Project Documentation

---

Last updated: July 2026

Purpose: Comprehensive reference enabling any engineer or AI assistant to understand, maintain, and extend this platform without prior context.

---

## Table of Contents

---

|                               |
|-------------------------------|
| Executive Summary             |
| Product Requirements          |
| User Roles & Permissions      |
| User Flows                    |
| Database Schema               |
| API Reference                 |
| AI Architecture               |
| Authentication & Security     |
| Admin System                  |
| Frontend                      |
| Third-Party Integrations      |
| Deployment                    |
| Known Issues & Technical Debt |
| Future Roadmap                |
| Architecture Diagram          |
| Folder Structure              |
| Development Notes             |
| README — Running Locally      |

---

## 1. Executive Summary

---

### Purpose

Athlete Intelligence is a B2B SaaS platform that gives sports agencies, sponsors, and national federations a real-time intelligence layer on their managed athletes. Rather than manually trawling sports media, social networks, and federation results pages, users add athletes by name and the platform's AI pipeline automatically researches, structures, and continuously surfaces intelligence across five categories: results & rankings, media & interviews, sponsorships, career changes, and competitive schedule.

### Target Users

- Sports managers

s  
m  
a  
n  
a  
gi  
n  
g  
ro  
st  
er  
s  
of  
5  
—  
5  
0  
+  
at  
hl  
et  
es  
•  
Br  
a  
n  
d/  
sp  
o  
ns  
or  
sh  
ip  
te  
a  
m  
s  
ev  
al  
u  
ati  
n  
g  
at  
hl  
et  
es  
fo  
r  
p  
ar  
tn  
er  
sh  
ip  
or  
m  
o  
nit  
or  
in  
g  
ex  
ist  
in  
g  
rel  
ati  
o

ns  
hi  
ps  
•  
N  
ati  
o  
n  
al  
sp  
or  
tin  
g  
fe  
d  
er  
ati  
o  
ns  
tr  
ac  
ki  
n  
g  
th  
eir  
hi  
g  
h-  
p  
er  
fo  
r  
m  
a  
nc  
e  
at  
hl  
et  
es  
• I  
n  
di  
vi  
d  
u  
al  
at  
hl  
et  
e  
m  
a  
n  
a  
g  
er  
s  
w  
h  
o  
n  
e  
e  
d  
a  
pr  
of

es  
si  
o  
n  
al  
int  
ell  
ig  
e  
nc  
e l  
ay  
er

## Core Value Proposition

Add an athlete by name — the platform does the rest. Within 60 seconds of creation, AI researchers (Perplexity Sonar Pro + GPT-4o) populate a complete dossier: biography, career timeline, competition history, sponsor relationships, management contacts, social media metrics, and a ranked intelligence feed. An AI chat assistant lets users ask natural-language questions across their full roster, grounded in the platform's database rather than LLM hallucination.

## Business Goals

- Monthly recurring revenue  
Subscription (Stripe)  
• Sales

-le  
d  
gr  
o  
wt  
h:  
d  
e  
m  
o  
e  
n  
q  
uir  
ie  
s  
vi  
a  
a  
co  
nt  
ac  
t f  
or  
m  
flo  
w  
int  
o  
a  
n  
a  
d  
mi  
n  
C  
R  
M  
•  
F  
o  
u  
n  
d  
er  
-  
m  
a  
n  
a  
g  
e  
d  
o  
p  
er  
ati  
o  
ns  
  
in  
th  
e  
e  
ar  
ly  
st  
a  
g

e  
—  
n  
o  
te  
a  
m  
-f  
ac  
in  
g  
a  
d  
mi  
n i  
nt  
er  
fa  
ce  
s r  
e  
q  
uir  
e  
d  
ye  
t  
b  
ey  
o  
n  
d  
th  
e  
fo  
u  
n  
d  
er'  
s  
a  
d  
mi  
n  
p  
a  
n  
el

## 2. Product Requirements

### Implemented Features

| Feature                          | Status     | Notes                                                   |
|----------------------------------|------------|---------------------------------------------------------|
| Athlete roster management (CRUD) | ' Complete | Add by name; AI auto-identifies sport/event/nationality |
| AI auto-populate pipeline        | ' Complete | Perplexity !' GPT-4o; populates all 5 related tables    |
| Intelligence feed per athlete    | ' Complete | Categorised, confidence-scored, source-attributed       |
|                                  |            |                                                         |

Timeline of career events

' Complete



|                                  |            |                                                      |
|----------------------------------|------------|------------------------------------------------------|
|                                  |            | Auto-populated; editable                             |
| Competition history & schedule   | ' Complete | Past results + upcoming events                       |
| Contact/network mapping          | ' Complete | Management, coaching, sponsorship contacts           |
| Social media metrics             | ' Complete | Handles + followers for Instagram, Twitter/X, TikTok |
| Per-athlete profile photo        | ' Complete | Wikipedia lookup with Perplexity fallback            |
| AI-generated dossier summary     | ' Complete | Streamed SSE; cached in DB                           |
| AI chat assistant                | ' Complete | Tool-calling agentic loop; database-grounded         |
| Athlete comparison               | ' Complete | Side-by-side stats for any two athletes              |
| Alert configuration              | ' Complete | Per-athlete, per-category notification preferences   |
| Stripe billing (3 tiers)         | ' Complete | Checkout, portal, webhook sync                       |
| Pricing page                     | ' Complete | Live prices from Stripe                              |
| Contact/demo enquiry form        | ' Complete | Rate-limited; stored in DB                           |
| Admin: customer list             | ' Complete | Clerk users × Stripe subscriptions                   |
| Admin: enquiry CRM               | ' Complete | Read, status updates                                 |
| Admin: bulk photo backfill       | ' Complete | Wikipedia + Perplexity per athlete                   |
| Admin: bulk social backfill      | ' Complete | Perplexity sonar-pro; 3 parallel                     |
| Admin: per-athlete repopulate    | ' Complete | Wipe + re-run full pipeline                          |
| Per-athlete social media refresh | ' Complete | Single athlete live lookup                           |
| Health check endpoint            | ' Complete | DB ping                                              |
| Landing page                     | ' Complete | Public marketing page                                |
| About / Security / Terms pages   | ' Complete | Public legal/marketing pages                         |
| Sign in / Sign up flow           | ' Complete | Clerk-managed                                        |
| Settings page                    | ' Complete | User preferences UI                                  |
| Sources page                     | ' Complete | Documents the AI's intelligence sources              |
|                                  |            |                                                      |



' Complete

Partially Implemented Features

| Feature            | Status      | Notes                                                      |
|--------------------|-------------|------------------------------------------------------------|
| Feed page          | & p Partial | UI exists; real-time push not connected                    |
| Schedule page      | & p Partial | UI exists; not linked to live calendar data                |
| Alerts delivery    | & p Partial | Config stored; no email/push delivery implemented          |
| New agent page     | & p Partial | Athlete creation wizard exists; onboarding flow incomplete |
| AI usage tracking  | & p Partial | Admin tab exists; no usage metering wired                  |
| Licence management | & p Partial | Admin tab exists; not functional                           |
| Feature flags      | & p Partial | Admin tab exists; no flag system implemented               |

Planned / Deferred Features

- Real-time creation workflow monitoring (live agent queue deployment, latency, technology)

e  
m  
et  
ry  
)  
•  
E  
m  
ail  
/p  
us  
h  
d  
eli  
ve  
ry  
fo  
r  
al  
er  
t  
n  
oti  
fic  
ati  
o  
ns  
•  
B  
ac  
kg  
ro  
u  
n  
d  
co  
nti  
nu  
o  
us  
cr  
a  
wl  
jo  
b  
(c  
ro  
n  
th  
at  
r  
e-  
ru  
ns  
a  
ut  
o-  
p  
o  
p  
ul  
at  
e  
fo  
r  
all  
a  
thl  
et

es  
)  
•  
Al  
u  
sa  
g  
e  
m  
et  
er  
in  
g  
+  
p  
er  
-u  
se  
r c  
re  
dit  
li  
mi  
ts  
•  
Te  
a  
m/  
m  
ult  
i-u  
se  
r  
ac  
co  
u  
nt  
s (  
cu  
rr  
e  
ntl  
y  
si  
n  
gl  
e-  
us  
er  
p  
er  
s  
u  
bs  
cri  
pti  
o  
n)  
•  
C  
S  
V  
/  
E  
xc  
el  
ex  
p  
or

t  
of  
a  
thl  
et  
e  
d  
os  
si  
er  
s

•  
C  
us  
to  
m  
at  
hl  
et  
e  
ta  
gs  
a  
n  
d  
p  
or  
tfo  
lio  
g  
ro  
u  
pi  
n  
gs

•  
W  
e  
b  
h  
o  
ok  
o  
ut  
b  
o  
u  
n  
d  
d  
eli  
ve  
ry  
fo  
r i  
nt  
ell  
ig  
e  
nc  
e  
ev  
e  
nt  
s

---

### 3. User Roles & Permissions

---

#### Visitor (unauthenticated)

- Can access : Landing page, Pricing page, About, Contact, Security, Terms, Sign In / Sign Up
- Cannot access



s:  
a  
ny  
/  
a  
p  
p/  
\*r  
o  
ut  
es  
,  
a  
ny  
p  
ro  
te  
ct  
e  
d  
A  
PI  
e  
n  
d  
p  
oi  
nt  
s  
•  
R  
at  
e-l  
im  
ite  
d  
o  
n:  
P  
O  
S  
T  
/a  
pi/  
co  
nt  
ac  
t(  
1  
0/  
h  
o  
ur  
p  
er  
l  
P)  
,  
P  
O  
S  
T  
/a  
pi/  
str  
ip  
e/  
ch  
ec

ko  
ut  
(  
1  
0/  
h  
o  
ur  
p  
er  
l  
P)

## Standard User (authenticated Clerk session)

- C  
a  
n  
ac  
ce  
ss  
: a  
ll  
a  
p  
p  
a  
g  
es

— D  
as  
h  
b  
o  
ar  
d,  
F  
e  
e  
d,  
D  
os  
si  
er  
,  
C  
o  
m  
p  
ar  
e,  
C  
h  
at  
,  
Al  
er  
ts,  
S  
ch  
e  
d  
ul  
e,  
S  
et

tings  
,  
Source  
s,  
Billing  
•  
Can  
call:  
all  
pro  
tect  
ed  
API  
end  
point  
s (re  
quir  
e Au  
th  
gate)  
•  
Can  
not  
ac  
cess:  
Ad  
mi  
n  
page,  
any  
/a  
pi/  
ad  
mi

n/  
\*  
e  
n  
d  
p  
oi  
nt  
s  
•  
N  
ot  
e:  
C  
ur  
re  
ntl  
y  
n  
o  
p  
er  
-u  
se  
r  
d  
at  
a i  
so  
lat  
io  
n  
—  
al  
l u  
se  
rs  
se  
e  
th  
e  
sa  
m  
e  
at  
hl  
et  
e  
ro  
st  
er  
. T  
hi  
s  
is  
a  
p  
pr  
o  
pr  
iat  
e  
fo  
r t  
h  
e  
a  
g  
e

ncy  
y  
us  
e-  
ca  
se  
(s  
h  
ar  
e  
d  
ro  
st  
er  
)  
bu  
t  
w  
o  
ul  
d  
n  
e  
e  
d  
ch  
a  
n  
gi  
n  
g  
fo  
r  
m  
ult  
i-t  
e  
n  
a  
nt  
S  
a  
a  
S.

### Administrator (Founder)

- I  
d  
e  
nti  
fie  
d  
by  
:  
Cl  
er  
k  
pr  
im  
ar  
y  
e  
m  
ail  
m  
at  
ch  
in  
g

a  
ni  
g  
e  
m  
mi  
ll  
@  
th  
e  
o  
ut  
si  
d  
ei  
n.  
nz  
(  
h  
ar  
dc  
o  
d  
e  
d  
in  
5  
pl  
ac  
es

—  
s  
e  
e  
§  
1  
3)  
•  
C  
a  
n  
ac  
ce  
ss  
:  
A  
d  
mi  
n  
p  
a  
g  
e  
at  
/  
a  
d  
mi  
n,  
al  
l/  
a  
pi/  
a  
d  
mi  
n/  
\*

e  
n  
d  
p  
oi  
nt  
s  
•  
A  
d  
mi  
n  
ca  
p  
a  
bil  
iti  
es  
: s  
e  
e  
s  
9

## No Other Roles

There is currently no team/manager hierarchy, no read-only viewer role, and no per-athlete access control.

---

## 4. User Flows

---

### 4.1 Sign Up

Visitor clicks "Get Started" or navigates to /sign-up  
Clerk-hosted sign-up modal (email + password, or Google OAuth)  
On completion, Clerk calls its webhook; the user is redirected to /app/dashboard  
No onboarding wizard — user lands directly on Dashboard

### 4.2 Sign In

User navigates to /sign-in  
Clerk-hosted sign-in modal  
On success, Clerk issues a session token; the React app's ClerkAuthSync component registers getToken with setAuthTokenGetter and registerTokenGetter  
All subsequent API calls from React Query hooks and manual authFetch calls include Authorization: Bearer <clerk-jwt>

### 4.3 Add an Athlete

User clicks "+ Add Athlete" in the sidebar or Dashboard  
NewAgentPage shown; user types athlete name and submits  
Frontend calls POST /api/athletes/discover with { name }  
Server: checks for duplicate !' calls discoverAthleteProfile (OpenAI GPT-4o) to identify sport/event/nationality !' inserts into athletes table !' fires autoPopulateAthlete in the background !' responds with athlete stub (201)  
Frontend navigates to the new athlete's dossier page  
Auto-populate runs asynchronously (~30–60s): Perplexity research !' GPT-4o extraction !' inserts intelligence, timeline, contacts, competitions, photo, social metrics  
Dossier page polls/refreshes showing data as it appears

### 4.4 View Athlete Dossier

User clicks an athlete in the sidebar or Dashboard  
Navigates to /app/dossier/:id  
Page fires 7 parallel API calls: GET /athletes/:id, /athletes/:id/intelligence, /athletes/:id/timeline, /athletes/:id/contacts, /athletes/:id/competitions, /athletes/:id/summary, /athletes/:id/alerts  
Data renders across tabs: Overview, Intelligence, Timeline, Competitions, Contacts, Social Media, Alerts  
AI Summary streams from GET /athletes/:id/summary (SSE); cached in DB after first generation

### 4.5 AI Chat

User navigates to /app/chat

Types a question about any athlete(s) on the roster

Frontend calls POST /api/chat — SSE stream

Backend: agentic tool-calling loop — up to 5 iterations:

- To  
ol  
ca  
lls  
e  
xe  
cu  
te  
d:  
g  
et  
  
—  
at  
hl  
et  
e  
  
—  
pr  
ofi  
le,  
g  
et  
  
—  
at  
hl  
et  
e  
\_i  
nt  
ell  
ig  
e  
nc  
e,  
g  
et  
  
—  
at  
hl  
et  
e  
\_c  
o  
m  
p  
eti  
tio  
ns  
,  
g  
et  
  
—  
at  
hl  
et  
e  
\_c  
o  
nt  
ac  
ts,  
s  
e  
ar



ch  
-  
at  
hl  
et  
es  
.  
P  
h  
as  
e l  
a  
b  
el  
s  
str  
e  
a  
m  
e  
d  
b  
ac  
k:  
"S  
e  
ar  
ch  
in  
g  
at  
hl  
et  
es  
...  
",  
"L  
o  
a  
di  
n  
g i  
nt  
ell  
ig  
e  
nc  
e  
...  
",  
et  
c.

Final analyst response streamed back token-by-token

Sources are cited from database; no LLM hallucination policy enforced via system prompt

## 4.6 Compare Athletes

User navigates to /app/compare

Selects two athletes from dropdowns

Frontend calls GET /api/athletes/compare?ids=1,2

Side-by-side stats rendered: rankings, PBs, social metrics, intelligence count

## 4.7 Configure Alerts

User opens athlete dossier !' Alerts tab

Per-category toggles (results, media, sponsorships, career) with frequency options (immediate, daily, weekly)

Frontend calls PUT /api/athletes/:id/alerts

Config saved to alert\_configs table

& p No delivery mechanism is currently implemented — config is stored but never actioned

## 4.8 Stripe Checkout (Subscribe)

User navigates to Pricing page  
Clicks a plan — frontend calls POST /api/stripe/checkout with { priceId, email, successUrl, cancelUrl, trialDays }  
Server creates/finds Stripe customer by email, creates Checkout Session  
User redirected to Stripe-hosted checkout  
On completion: redirected to /billing-success page  
Stripe sends webhook !' WebhookHandlers.processWebhook !' stripe-replit-sync writes subscription data into stripe.\* schema tables  
GET /api/stripe/subscription now returns active subscription

## 4.9 Access Billing Portal

Authenticated user navigates to Settings !' Billing  
Frontend calls POST /api/stripe/portal with { returnUrl }  
Server looks up Stripe customer by email !' creates billing portal session  
User redirected to Stripe portal (cancel, upgrade, update payment)

## 4.10 Admin — Customer Review

Founder navigates to /admin  
Customers tab: calls GET /api/admin/customers — merges Clerk user list with Stripe subscription data  
Enquiries tab: calls GET /api/admin/enquiries — shows all contact form submissions with status  
Can mark enquiries as read/replied via PUT /api/admin/enquiries/:id

## 4.11 Admin — Intelligence Repopulate

Founder navigates to /admin !' Crawl tab  
"Repopulate athlete intelligence" panel shows all athletes with item counts and last crawl date  
Athlete with 0 items shown with red count  
Clicking "Repopulate" calls POST /api/admin/repopulate/:id — clears all existing data, fires autoPopulateAthlete in background, returns 202  
Panel polls GET /api/athletes/:id every 4s until lastCrawledAt is set, then shows "Done — N items"

# 5. Database Schema

All tables use PostgreSQL via Drizzle ORM. Connection: DATABASE\_URL environment variable (PostgreSQL connection string). The stripe.\* schema is a separate Stripe-synced schema managed by the stripe-replit-sync integration.

### `athletes`

Core athlete profiles. One row per tracked athlete.

| Column      | Type        | Nullable | Default | Description                                   |
|-------------|-------------|----------|---------|-----------------------------------------------|
| id          | serial (PK) | No       | auto    | Primary key                                   |
| name        | text        | No       | —       | Full name                                     |
| sport       | text        | No       | —       | Sport category (e.g. "Athletics", "Cycling")  |
| event       | text        | No       | —       | Specific event (e.g. "High Jump", "Downhill") |
| nationality | text        | No       | —       | Country name                                  |
| age         | integer     | Yes      | null    | Age in years                                  |
| squad       | text        | No       | ""      | Current squad/team                            |
|             |             |          |         |                                               |

world\_rank

integer

Yes

null

|                         |             |     |          |                                             |
|-------------------------|-------------|-----|----------|---------------------------------------------|
|                         |             |     |          | Current world ranking                       |
| world_rank_delta        | integer     | No  | 0        | Change in world rank (positive = improved)  |
| national_rank           | integer     | Yes | null     | National ranking                            |
| personal_best           | text        | Yes | null     | Personal best result string                 |
| season_best             | text        | Yes | null     | Season best result string                   |
| instagram_handle        | text        | Yes | null     | Instagram username (no @)                   |
| instagram_followers     | integer     | No  | 0        | Instagram follower count                    |
| instagram_engagement    | real        | No  | 0        | Engagement rate (%)                         |
| twitter_handle          | text        | Yes | null     | Twitter/X username (no @)                   |
| twitter_followers       | integer     | No  | 0        | Twitter follower count                      |
| tiktok_handle           | text        | Yes | null     | TikTok username (no @)                      |
| tiktok_followers        | integer     | No  | 0        | TikTok follower count                       |
| follower_growth_30d     | real        | No  | 0        | 30-day follower growth %                    |
| avg_engagement          | real        | No  | 0        | Average engagement rate across platforms    |
| agent_status            | text        | No  | "active" | Monitoring status: active \                 |
| last_crawled_at         | timestamptz | Yes | null     | Last auto-populate completion time          |
| intelligence_count      | integer     | No  | 0        | Cached count of intelligence items          |
| has_new_intelligence    | boolean     | No  | false    | Flag for unread new intelligence            |
| avatar_url              | text        | Yes | null     | Profile photo URL (Wikipedia or Perplexity) |
| ai_summary              | text        | Yes | null     | Cached AI-generated dossier summary         |
| ai_summary_generated_at | timestamptz | Yes | null     | When the cached summary was generated       |
| created_at              | timestamptz | No  | now()    | Creation timestamp                          |
| updated_at              | timestamptz | No  | now()    | Last update timestamp                       |

paused \

Indexes: Primary key on id.  
Relationships: Parent to intelligence\_items, timeline\_events, contacts, competitions, alert\_configs (all cascade delete).

``intelligence_items``

Structured intelligence derived from the AI research pipeline. One row per discrete intelligence item.

| Column        | Type         | Nullable | Default | Description                             |
|---------------|--------------|----------|---------|-----------------------------------------|
| id            | serial (PK)  | No       | auto    | Primary key                             |
| athlete_id    | integer (FK) | No       | —       | !' athletes.id (cascade delete)         |
| athlete_name  | text         | No       | —       | Denormalised name for display           |
| category      | text         | No       | —       | results_rankings \media_interviews \    |
| title         | text         | No       | —       | Short headline                          |
| summary       | text         | Yes      | null    | 1–3 sentence summary                    |
| source_domain | text         | No       | —       | Source domain (e.g. worldathletics.org) |
| source_url    | text         | Yes      | null    | Full source URL if known                |
| confidence    | integer      | No       | 80      | AI confidence score 0–100               |
| published_at  | timestampz   | Yes      | null    | When the event/article was published    |
| discovered_at | timestampz   | No       | now()   | When the platform discovered it         |

``timeline_events``

Career milestone events for each athlete.

| Column     | Type         | Nullable | Default | Description                     |
|------------|--------------|----------|---------|---------------------------------|
| id         | serial (PK)  | No       | auto    | Primary key                     |
| athlete_id | integer (FK) | No       | —       | !' athletes.id (cascade delete) |
| date       | date         | No       | —       | Event date (YYYY-MM-DD string)  |
| category   | text         | No       | —       | competition \media \            |
| title      | text         | No       | —       | Short event name                |
|            |              |          |         |                                 |



description

text

Yes

null

|               |         |     |       |                                   |
|---------------|---------|-----|-------|-----------------------------------|
|               |         |     |       | Longer description                |
| location      | text    | Yes | null  | Venue / city                      |
| source_domain | text    | No  | —     | Source domain                     |
| source_url    | text    | Yes | null  | Full source URL                   |
| confidence    | integer | No  | 85    | AI confidence score 0–100         |
| significant   | boolean | No  | false | Whether this is a major milestone |

## `competitions`

Historical results and upcoming competition schedule.

| Column       | Type         | Nullable | Default    | Description                                       |
|--------------|--------------|----------|------------|---------------------------------------------------|
| id           | serial (PK)  | No       | auto       | Primary key                                       |
| athlete_id   | integer (FK) | No       | —          | ! athletes.id (cascade delete)                    |
| athlete_name | text         | No       | —          | Denormalised name                                 |
| meet_name    | text         | No       | —          | Competition name                                  |
| event        | text         | No       | —          | Specific event competed (e.g. "100m", "Downhill") |
| location     | text         | Yes      | null       | Venue / city                                      |
| date         | date         | No       | —          | Competition date (YYYY-MM-DD string)              |
| tier         | text         | No       | "B"        | Competition tier: A (World Champs/Olympics) \     |
| status       | text         | No       | "upcoming" | upcoming \                                        |
| result       | text         | Yes      | null       | Result string (e.g. "1st, 2.36m", "DNF")          |
| created_at   | timestampz   | No       | now()      | Creation timestamp                                |

## `contacts`

Known contacts in an athlete's professional network.

| Column | Type        | Nullable | Default | Description |
|--------|-------------|----------|---------|-------------|
| id     | serial (PK) | No       | auto    | Primary key |
|        |             |          |         |             |

athlete\_id

integer (FK)

No





|                 |         |     |            |                                               |               |
|-----------------|---------|-----|------------|-----------------------------------------------|---------------|
|                 |         |     |            | !' athletes.id (cascade delete)               |               |
| role            | text    | No  | —          | Job title / role (e.g. "Head Coach", "Agent") |               |
| category        | text    | No  | —          | management \                                  | coaching \    |
| name            | text    | No  | —          | Full name                                     |               |
| org             | text    | No  | —          | Organisation name                             |               |
| org_type        | text    | Yes | null       | Organisation type                             |               |
| status          | text    | No  | "verified" | verified \                                    | unconfirmed \ |
| confidence      | integer | No  | 80         | AI confidence 0–100                           |               |
| public_email    | text    | Yes | null       | Publicly available email                      |               |
| website         | text    | Yes | null       | Organisation or personal website              |               |
| note            | text    | Yes | null       | Contextual note                               |               |
| last_verified   | date    | No  | —          | Last verification date                        |               |
| date_discovered | date    | No  | —          | When first found                              |               |
| source_domain   | text    | No  | —          | Source domain                                 |               |
| source_excerpt  | text    | Yes | null       | Quote from the source mentioning this contact |               |

### `alert\_configs`

Per-athlete notification preferences. One row per athlete (1:1 with athletes).

| Column               | Type             | Nullable | Default     | Description                                  |         |
|----------------------|------------------|----------|-------------|----------------------------------------------|---------|
| athlete_id           | integer (PK, FK) | No       | —           | !' athletes.id (cascade delete); also the PK |         |
| results_enabled      | boolean          | No       | true        | Results & rankings alerts on/off             |         |
| results_frequency    | text             | No       | "immediate" | immediate \                                  | daily \ |
| media_enabled        | boolean          | No       | true        | Media alerts on/off                          |         |
| media_frequency      | text             | No       | "daily"     | immediate \                                  | daily \ |
| sponsorships_enabled | boolean          | No       | true        | Sponsorship alerts on/off                    |         |

sponsorships\_frequency

text

No

"daily"

immediate \





|                  |         |    |             |                             |         |
|------------------|---------|----|-------------|-----------------------------|---------|
| career_enabled   | boolean | No | true        | Career change alerts on/off |         |
| career_frequency | text    | No | "immediate" | immediate \                 | daily \ |

### `contact\_enquiries`

Demo / sales / general enquiries from the public contact form.

| Column     | Type        | Nullable | Default   | Description                               |         |
|------------|-------------|----------|-----------|-------------------------------------------|---------|
| id         | serial (PK) | No       | auto      | Primary key                               |         |
| type       | text        | No       | "general" | demo \                                    | sales \ |
| name       | text        | No       | —         | Submitter's full name                     |         |
| org        | text        | No       | —         | Organisation                              |         |
| email      | text        | No       | —         | Contact email (stored lowercase)          |         |
| role       | text        | Yes      | null      | Job title                                 |         |
| athletes   | text        | Yes      | null      | Roster size estimate (for demo enquiries) |         |
| message    | text        | Yes      | null      | Free-text message                         |         |
| status     | text        | No       | "new"     | new \                                     | read \  |
| created_at | timestamptz | No       | now()     | Submission timestamp                      |         |

### `conversations`

AI chat conversation sessions.

| Column     | Type        | Nullable | Default | Description        |
|------------|-------------|----------|---------|--------------------|
| id         | serial (PK) | No       | auto    | Primary key        |
| title      | text        | No       | —       | Conversation title |
| created_at | timestamptz | No       | now()   | Creation timestamp |

### `messages`

Individual messages within a chat conversation.

| Column | Type | Nullable | Default | Description |
|--------|------|----------|---------|-------------|
|--------|------|----------|---------|-------------|



serial (PK)

No

auto

|                 |              |    |       |                                     |
|-----------------|--------------|----|-------|-------------------------------------|
| Primary key     |              |    |       |                                     |
| conversation_id | integer (FK) | No | —     | ! conversations.id (cascade delete) |
| role            | text         | No | —     | user \ assistant \                  |
| content         | text         | No | —     | Message content                     |
| created_at      | timestampz   | No | now() | Message timestamp                   |

### `stripe.\*` schema (managed by stripe-replit-sync)

The Stripe integration maintains a shadow schema called stripe containing synced tables: stripe.products, stripe.prices, stripe.customers, stripe.subscriptions. These are queried directly via raw SQL in stripeStorage.ts. They are not managed by Drizzle migrations — they are created and updated by the stripe-replit-sync package's webhook handler.

## 6. API Reference

Base URL: /api

Authentication: All protected routes require Authorization: Bearer <clerk-jwt> header. The token is obtained via useAuth().getToken() from @clerk/react.

### Public Endpoints (no auth required)

#### `GET /api/healthz`

Health check. Verifies database connectivity.

•

```

R
es
p
o
ns
e
2
0:
{ "
st
at
us
":
"o
k"
}

```

•

```

R
es
p
o
ns
e
5
0:
3:
{ "
st
at
us

```

":  
"e  
rr  
or  
",  
"r  
e  
as  
o  
n"  
:"  
d  
at  
a  
b  
as  
e  
u  
n  
av  
ail  
a  
bl  
e"  
}

**`POST /api/contact`**

Submit a contact/demo enquiry.

•

R  
at  
e l  
im  
it:  
1  
0  
re  
q  
u  
es  
ts  
p  
er  
l  
P  
p  
er  
h  
o  
ur

•

B  
o  
dy  
:{  
ty  
p  
e,  
n  
a  
m  
e,  
or  
g,  
e  
m  
ail  
, r  
ol

e,  
at  
hl  
et  
es  
,  
m  
es  
sa  
g  
e  
}(  
=  
r  
e  
q  
uir  
e  
d)  
•  
R  
es  
p  
o  
ns  
e  
2  
0  
0:  
{ "  
ok  
":  
tr  
u  
e,  
"i  
d"  
: <  
e  
n  
q  
uir  
y\_  
id  
>  
}  
•  
R  
es  
p  
o  
ns  
e  
4  
0  
0:  
{ "  
er  
ro  
r":  
"  
n  
a  
m  
e,  
or  
g,  
a  
n



d  
e  
m  
ail  
a  
re  
r  
e  
q  
uir  
e  
d"  
}

**`GET /api/stripe/prices`**

Fetch active Stripe price IDs (for the pricing page).

•  
R  
es  
p  
o  
ns  
e  
2  
0:  
{ "  
pr  
ic  
es  
":  
{ "  
st  
ar  
te  
r":  
<  
pr  
ic  
el  
d  
>,  
"  
pr  
o"  
:  
<  
pr  
ic  
el  
d  
>,  
"  
e  
nt  
er  
pr  
is  
e"  
:  
<  
pr  
ic  
el  
d  
>  
}}}

**`GET /api/stripe/products-with-prices`**

Full product catalogue with prices.

•

Response:  
{"products": [{"id": "name", "description": "..."}]}

**POST /api/stripe/checkout`**  
Create a Stripe Checkout Session.

• Rate limit:  
10 requests per IP per hour.  
• Body

: {  
p  
ric  
el  
d,  
s  
uc  
ce  
ss  
Ur  
l,  
ca  
nc  
el  
Ur  
l,  
e  
m  
ail  
, t  
ri  
al  
D  
ay  
s  
}  
(  
=  
r  
e  
q  
uir  
e  
d)  
•  
R  
es  
p  
o  
ns  
e  
2  
0  
0:  
{ "  
ur  
l":  
"<  
str  
ip  
e  
ch  
ec  
ko  
ut  
u  
rl  
>"  
}  
•  
R  
es  
p  
o  
ns  
e  
4  
0  
0:  
{ "

```
error": {
  price_id,
  success_url,
  and_cancel_url,
  required"
}
```

#### `POST /api/stripe/webhook``

Stripe webhook receiver. Receives raw Buffer — must be registered before `express.json()`.

- Header required: stripe-signature
- Response: 200: {"receiver

```
d"
: t
ru
e
}
```

---

## Protected Endpoints (require Clerk Bearer token)

### Dashboard

##### GET /api/dashboard

Returns overview stats and priority athlete data.

```
•
R
es
p
o
ns
e
2
0
0:
  ``json
  {
    "totalAthletes": 5,
    "activeAgents": 5,
    "newIntelligence": 2,
    "upcomingCompetitions": 3,
    "priorityAlerts": 2,
    "recentIntelligence": [...],
    "priorityAthletes": [...]
  }
  ...
```

### Athletes

##### GET /api/athletes

List all athletes ordered by name.

```
•
R
es
p
o
ns
e
2
0
0:
Ar
ra
y
of
a
thl
et
e
o
bj
ec
ts
```

##### POST /api/athletes/discover

Create an athlete by name; AI auto-identifies profile.

```
•
B
o
dy
: {
  "
```

name": "Brooklyn Macdonald"}  
• Responses 201: {"athlete": {"..."}, "created": true}  
} (or 200 with "create": false if exists)  
• Side effect:

Fi  
re  
s  
a  
ut  
o  
P  
o  
p  
ul  
at  
e  
At  
hl  
et  
e  
in  
b  
ac  
kg  
ro  
u  
n  
d

##### POST /api/athletes/bulk  
Create multiple athletes in one call.

- B  
o  
dy  
: {  
"  
at  
hl  
et  
es  
":  
[ {  
"n  
a  
m  
e"  
, "  
sp  
or  
t", "  
ev  
e  
nt  
", "  
"n  
ati  
o  
n  
ali  
ty  
"}  
,  
..  
..]  
}  
•

R  
es  
p  
o  
ns  
e  
2

0  
1:  
Ar  
ra  
y  
of  
cr  
e  
at  
e  
d  
at  
hl  
et  
es

##### GET /api/athletes/compare  
Compare two athletes by ID.

•  
Q  
u  
er  
y  
p  
ar  
a  
m  
s:  
id  
s=  
1,  
2

•  
R  
es  
p  
o  
ns  
e  
2  
0:  
{ "  
at  
hl  
et  
es  
":  
[..  
.t  
w  
o  
at  
hl  
et  
e  
o  
bj  
ec  
ts.  
..]  
}

##### GET /api/athletes/:id  
Fetch a single athlete.

•  
R  
es  
p  
o



ns  
e  
2  
0  
0:  
At  
hl  
et  
e  
o  
bj  
ec  
t  
•

R  
es  
p  
o  
ns  
e  
4  
0  
4:  
{ "  
er  
ro  
r":  
"

At  
hl  
et  
e  
n  
ot  
fo  
u  
n  
d"  
}

##### PATCH /api/athletes/:id  
Update athlete fields.

•  
B  
o  
dy  
:  
P  
ar  
tia  
l a  
thl  
et  
e  
o  
bj  
ec  
t (  
a  
ny  
u  
p  
d  
at  
a  
bl  
e  
fie  
ld  
s)

- Res  
p  
o  
ns  
e  
2  
0  
0:  
U  
p  
d  
at  
e  
d  
at  
hl  
et  
e  
o  
bj  
ec  
t

##### DELETE /api/athletes/:id  
Delete an athlete and all related data (cascade).

- Res  
p  
o  
ns  
e  
2  
0  
4:  
N  
o  
co  
nt  
e  
nt

- Res  
p  
o  
ns  
e  
4  
0  
4:  
{ "  
er  
ro  
r":  
"  
At  
hl  
et  
e  
n  
ot  
fo  
u  
n  
d"  
}

##### POST /api/athletes/:id/repopulate

Wipe and re-run the full intelligence pipeline for one athlete.

- 

R  
es  
p  
o  
ns  
e  
2  
0  
2:  
{ "  
m  
es  
sa  
g  
e"  
: "  
R  
e-  
p  
o  
p  
ul  
ati  
o  
n  
st  
ar  
te  
d  
fo  
r  
<  
n  
a  
m  
e  
>"  
}

- 

Si  
d  
e  
ef  
fe  
ct:  
C  
le  
ar  
s  
all  
r  
el  
at  
e  
d  
ta  
bl  
es  
, fi  
re  
s  
a  
ut  
o  
P  
o

p  
ul  
at  
e  
At  
hl  
et  
e  
in  
b  
ac  
kg  
ro  
u  
n  
d

##### POST /api/athletes/:id/refresh-social

Run a live Perplexity social media lookup for one athlete and update the DB.

•  
R  
es  
p  
o  
ns  
e  
2  
0:  
{ "  
ok  
":  
tr  
u  
e,  
"u  
p  
d  
at  
e  
d"  
: t  
ru  
e,  
"d  
at  
a"  
: {  
...  
},  
"r  
es  
e  
ar  
ch  
S  
u  
m  
m  
ar  
y"  
: "  
...  
"}  
"

## Intelligence

##### GET /api/athletes/:id/intelligence

Get all intelligence items for an athlete, ordered newest first.

•

Response 200: { "items": [..] }

##### GET /api/intelligence  
Get intelligence across all athletes (used for Feed page).

•

Response 200: Array of intelligence items with athlete info

## Competitions

##### GET /api/athletes/:id/competitions  
Get competition history for an athlete.

•

Response 200: { "competi

```
tio
ns
":
[.
.]
}
```

## Timeline

##### GET /api/athletes/:id/timeline  
Get timeline events for an athlete.

```
•
R
es
p
o
ns
e
2
0:
{ "
ev
e
nt
s"
: [
..
]
}
```

## Contacts

##### GET /api/athletes/:id/contacts  
Get network contacts for an athlete.

```
•
R
es
p
o
ns
e
2
0:
{ "
co
nt
ac
ts
":
[.
.]
}
```

## Alerts

##### GET /api/athletes/:id/alerts  
Get alert config for an athlete. Auto-creates default config if missing.

```
•
R
es
p
o
ns
e
2
0:
Al
er
tC
```

o  
nfi  
g  
o  
bj  
ec  
t

##### PUT /api/athletes/:id/alerts  
Update alert config.

•

B  
o  
dy  
:  
P  
ar  
tia  
l  
Al  
er  
tC  
o  
nfi  
g  
(a  
ny  
fi  
el  
ds  
)

•

R  
es  
p  
o  
ns  
e  
2  
0  
0:  
U  
p  
d  
at  
e  
d  
Al  
er  
tC  
o  
nfi  
g

## AI Features

##### GET /api/athletes/:id/summary  
Stream an AI-generated dossier summary via SSE.

•

R  
es  
p  
o  
ns  
e:  
te  
xt/  
ev  
e  
nt

-st  
re  
a  
m  
•  
S  
S  
E  
ev  
e  
nt  
s:  
{ "  
to  
ke  
n"  
: "  
...  
"}  
(s  
tr  
e  
a  
mi  
n  
g)  
, {  
,"  
d  
o  
n  
e"  
: t  
ru  
e  
}(  
co  
m  
pl  
et  
e)  
, {  
,"  
er  
ro  
r":  
,"  
..."  
}  
(f  
ail  
ur  
e)  
•  
C  
ac  
hi  
n  
g:  
R  
es  
ult  
st  
or  
e  
d  
in  
at  
hl



et  
es  
.ai  
\_s  
u  
m  
m  
ar  
y;  
re  
-g  
e  
n  
er  
at  
e  
d  
if  
ol  
d  
er  
t  
h  
a  
n  
2  
4  
h

##### POST /api/chat  
Stream an AI analyst response via SSE.

•  
R  
at  
e l  
im  
it:  
2  
0  
re  
q  
u  
es  
ts  
p  
er  
1  
5  
mi  
nu  
te  
s  
p  
er  
u  
se  
r

•  
B  
o  
dy  
: {  
"  
m  
es  
sa  
g  
es  
":  
[{

"r  
ol  
e"  
:"  
us  
er  
"l"  
as  
si  
st  
a  
nt  
,"  
"c  
o  
nt  
e  
nt  
":  
":  
":  
":  
"},  
"  
at  
hl  
et  
el  
ds  
"?  
: [.  
..]  
}  
•  
R  
es  
p  
o  
ns  
e:  
te  
xt/  
ev  
e  
nt  
-st  
re  
a  
m  
•  
S  
S  
E  
ev  
e  
nt  
s:  
{ "  
p  
h  
as  
e"  
:"  
...  
"}  
(  
pr  
o  
gr  
es

```
s),
{
  "t
ok
e
n"
: "
...
"}
(s
tr
e
a
mi
n
n
g)
,{
"
d
o
n
e"
:t
ru
e
},
{"
er
ro
r":
"
..
"}
}
```

### Stripe (Protected)

##### POST /api/stripe/portal  
Create a Stripe Customer Portal session.

```
•
B
o
dy
:{
  "r
et
ur
n
Ur
l":
"h
tt
ps
://
...
"}
•
R
es
p
o
ns
e
2
0
0:
{"
ur
l":
"<
str
```

```
ip
e
p
or
tal
u
rl
>"
}
##### GET /api/stripe/subscription
Get the current user's subscription status.
•
R
es
p
o
ns
e
2
0
0:
{"
su
bs
cri
pti
o
n"
: <
St
ri
p
e
su
bs
cri
pti
o
n
o
bj
ec
t
or
n
ull
>
}
```

---

## Admin Endpoints (require Clerk Bearer + founder email)

All admin endpoints use the same auth flow as protected endpoints, plus an additional email check against `FOUNDER_EMAIL = "anigemmill@theoutsidein.nz"`.

##### GET /api/admin/customers  
List all Clerk users merged with their Stripe subscription data.

```
•
R
es
p
o
ns
e
2
0
0:
{"
cu
```

```
st
o
m
er
s"
: [
{
"
id
",
"n
a
m
e"
,
"
e
m
ail
",
"p
la
n"
,
"
su
bs
cri
pti
o
n
St
at
us
",
"
m
rr"
,
"
tri
al
E
n
d"
,
"
si
g
n
e
d
U
p
At
",
"l
as
tA
cti
ve
At
",
"i
m
a
g
e
Ur
l"
}]
}
```

##### GET /api/admin/enquiries  
List all contact enquiries ordered newest first.

- Response 200: { "enquiries": [ .. ] }

##### PUT /api/admin/enquiries/:id  
Update an enquiry's status.

- Body: { "status": "new" | "ready" | "replied" }

- Response 200: Updated enquiry

##### POST /api/admin/backfill-photos

Run Wikipedia/Perplexity photo lookup for all athletes missing a photo.

- 

R  
es  
p  
o  
ns  
e  
2  
0  
0:  
{ "  
ok  
":  
tr  
u  
e,  
"t  
ot  
al  
":  
N,  
"f  
o  
u  
n  
d"  
:  
N,  
"  
sk  
ip  
p  
e  
d"  
:  
N  
}

##### POST /api/admin/backfill-social

Run live Perplexity social media lookup for all athletes (3 in parallel).

- 

R  
es  
p  
o  
ns  
e  
2  
0  
0:  
{ "  
ok  
":  
tr  
u  
e,  
"t  
ot  
al  
":  
N,  
"  
u  
p  
d  
at  
e  
d"

:  
N,  
"  
n  
ot  
F  
o  
u  
n  
d"  
:  
N,  
"r  
es  
ult  
s"  
:  
[.  
..]  
}  
•  
N  
ot  
e:  
re  
s.  
se  
tTi  
m  
e  
o  
ut  
(0  
)  
is  
ca  
lle  
d  
at  
t  
h  
e  
st  
ar  
t  
of  
t  
hi  
s  
h  
a  
n  
dl  
er  
t  
o  
di  
sa  
bl  
e  
th  
e  
gl  
o  
b  
al  
3  
0-  
se  
co



nd  
tim  
e  
o  
ut

##### POST /api/admin/repopulate/:id

Wipe and re-run the full intelligence pipeline for one athlete (admin-triggered).

- 

Res  
po  
ns  
e  
2  
0  
2:  
{ "  
mes  
sa  
ge"  
: "  
Re-  
pop  
ulat  
ion  
start  
ed  
for  
<  
nam  
e"  
>  
}

---

## 7. AI Architecture

---

### Overview

The AI layer uses a two-provider strategy:

- 

Per  
plex  
ity  
So  
n  
ar  
P  
ro

(via Open Router) for live web search

— used whenever needed

• OpenAI GPT-4o / GPT-4o-mini

ni  
(vi  
a  
R  
e  
pli  
t  
Al  
I  
nt  
e  
gr  
ati  
o  
ns  
p  
ro  
xy  
) f  
or  
st  
ru  
ct  
ur  
e  
d  
ex  
tr  
ac  
tio  
n,  
s  
u  
m  
m  
ar  
is  
ati  
o  
n,  
a  
n  
d  
ch  
at

—  
u  
se  
d  
w  
h  
e  
n  
re  
as  
o  
ni  
n  
g  
ov  
er  
r  
et  
ri  
ev  
e  
d  
d

at  
a

Both providers are accessed via Replit's AI Integrations proxy (localhost:1106/modelfarm/\*), so no API keys are needed in application code — the proxy handles authentication.

---

## Auto-Populate Pipeline (`auto-populate.ts`)

Triggered by: POST /athletes/discover, POST /athletes/bulk, POST /athletes/:id/repopulate, POST /admin/repopulate/:id

Phase 1 — Perplexity web research

- M  
o  
d  
el:  
p  
er  
pl  
ex  
ity  
/s  
o  
n  
ar  
vi  
a  
O  
p  
e  
n  
R  
o  
ut  
er
- I  
n  
p  
ut  
:  
At  
hl  
et  
e  
n  
a  
m  
e,  
sp  
or  
t,  
ev  
e  
nt  
,  
n  
ati  
o  
n  
ali  
ty
- Pr  
o  
m  
pt  
:  
C

o  
m  
pr  
e  
h  
e  
ns  
iv  
e  
re  
se  
ar  
ch  
c  
ov  
er  
in  
g  
cu  
rr  
e  
nt  
st  
at  
us  
, r  
es  
ult  
s  
2  
0  
1  
6  
–  
2  
0  
2  
6,  
c  
ar  
e  
er  
ti  
m  
eli  
n  
e,  
ra  
nk  
in  
gs  
, s  
oc  
ial  
m  
e  
di  
a,  
c  
o  
nt  
ac  
ts,  
s  
p  
o  
ns  
or  
s,

inj  
ur  
y  
hi  
st  
or  
y  
•  
O  
ut  
p  
ut  
:  
Fr  
e  
e-  
te  
xt  
re  
se  
ar  
ch  
s  
u  
m  
m  
ar  
y  
wi  
th  
ci  
te  
d  
so  
ur  
ce  
s

Phase 2 — Twitter/X follower enrichment

•  
M  
o  
d  
el:

X  
A  
PI  
v  
2  
(/  
2/  
us  
er  
s/  
by  
/u  
se  
rn  
a  
m  
e/  
:h  
a  
n  
dl  
e  
?  
us  
er

.fi  
el  
ds  
=  
p  
u  
bli  
c\_  
m  
et  
ric  
s)  
•  
C  
o  
n  
dit  
io  
n:  
O  
nl  
y r  
u  
ns  
if  
T  
W  
IT  
T  
E  
R  
  
\_B  
E  
A  
R  
E  
R  
  
\_T  
O  
K  
E  
N  
e  
nv  
v  
ar  
  
is  
se  
t  
•  
E  
xtr  
ac  
ts:  
R  
e  
al-  
ti  
m  
e  
fol  
lo  
w  
er  
c  
o

unt  
for  
any  
discover  
ed  
Twitter  
her  
hand  
le

Phase 3 — Photo lookup

•

C  
all  
sf  
etch  
Wiki  
pedi  
a  
Photo  
on  
ame,  
sports  
from  
m  
phot  
ol  
ok  
upt  
s

•

Tr  
es  
:e  
xact  
Wiki  
pedi  
a



p  
a  
g  
e  
!'  
W  
iki  
p  
e  
di  
a  
se  
ar  
ch  
(  
n  
a  
m  
e-  
m  
at  
ch  
e  
d)  
!'  
P  
er  
pl  
ex  
ity  
i  
m  
a  
g  
e  
se  
ar  
ch  
•  
N  
a  
m  
e-  
m  
at  
ch  
g  
u  
ar  
d:  
W  
iki  
p  
e  
di  
a  
se  
ar  
ch  
r  
es  
ult  
s  
m  
us  
t c  
o  
nt  
ai

n  
at  
hl  
et  
e  
fir  
st  
A  
N  
D  
la  
st  
n  
a  
m  
e  
in  
th  
e  
titl  
e  
to  
b  
e  
ac  
ce  
pt  
e  
d  
(p  
re  
ve  
nt  
s  
wr  
o  
n  
g-  
p  
er  
so  
n  
m  
at  
ch  
es  
)

Phase 4 — OpenAI structured extraction

•  
M  
o  
d  
el:  
g  
pt  
-4  
o  
vi  
a  
O  
p  
e  
n  
AI  
I  
nt  
e  
gr  
ati

on  
• Input:  
The helpful  
Perplexity  
res  
earch  
text

• Prompt:  
Instructions  
extraction  
of

a  
structured  
JSON  
object  
with  
keys:

• at  
hl  
et  
e  
—  
r  
a

nk  
in  
gs  
,  
P  
B  
S,  
so  
ci  
al  
h  
a  
n  
dl  
es  
/fo  
llo  
w  
er  
s,  
sq  
u  
a  
d  
• i  
nt  
ell  
ig  
e  
nc  
el  
te  
m  
s  
—  
a  
rr  
ay  
o  
f  
5  
—  
1  
5 i  
nt  
ell  
ig  
e  
nc  
e i  
te  
m  
s  
• t  
im  
eli  
n  
e  
E  
ve  
nt  
s  
—  
a  
rr  
ay  
o  
f c  
ar

e  
er  
e  
ve  
nt  
s

•  
co  
nt  
ac  
ts

—  
a  
rr  
ay  
o  
f  
n  
et  
w  
or  
k  
co  
nt  
ac  
ts

•  
co  
m  
p  
eti  
tio  
ns

—  
a  
rr  
ay  
o  
f  
hi  
st  
or  
ic  
al/  
u  
pc  
o  
mi  
n  
g  
co  
m  
p  
eti  
tio  
ns

•  
R  
es  
p  
o  
ns  
e  
fo  
r  
m  
at  
:j

so  
n  
—  
o  
bj  
ec  
t  
•  
V  
ali  
d  
ati  
o  
n:  
E  
ac  
h  
ar  
ra  
y  
el  
e  
m  
e  
nt  
v  
ali  
d  
at  
e  
d  
b  
ef  
or  
e  
D  
B  
in  
se  
rt;  
m  
alf  
or  
m  
e  
d  
e  
nt  
ri  
es  
s  
ki  
p  
p  
e  
d  
wi  
th  
  
a  
w  
ar  
ni  
n  
g  
Phase 5 — DB writes  
•  
All

5  
ta  
bl  
es  
u  
p  
d  
at  
e  
d:  
at  
hl  
et  
es  
(  
p  
at  
ch  
) , i  
nt  
ell  
ig  
e  
nc  
e  
\_it  
e  
m  
s,  
ti  
m  
eli  
n  
e  
\_ev  
e  
nt  
s,  
co  
nt  
ac  
ts,  
c  
o  
m  
p  
eti  
tio  
ns  
•  
At  
hl  
et  
e'  
s l  
as  
tC  
ra  
wl  
e  
d  
At  
u  
p  
d  
at  
e  
d;

intelligence Consultant's  
Consultant's  
Confidential

Error handling: Entire pipeline is wrapped in try/catch. Failure is non-fatal — athlete row already exists; error is logged. Auto-populate never surfaces errors to the API caller.

## Chat Pipeline (`chat.ts`)

Model: gpt-4o via OpenAI Integration  
Transport: SSE (Server-Sent Events)  
Pattern: Agentic tool-calling loop (up to 5 iterations)

Tools available:

| Tool name                | What it does                                         |
|--------------------------|------------------------------------------------------|
| get_athlete_profile      | Full athlete row by ID                               |
| get_athlete_intelligence | All intelligence items for athlete                   |
| get_athlete_competitions | Competition history for athlete                      |
| get_athlete_contacts     | Contact network for athlete                          |
| search_athletes          | Search roster by name                                |
| get_all_athletes         | Full roster (for queries spanning multiple athletes) |

Loop: System prompt grounds the model in database-first analysis. The model calls tools, the server executes them (direct DB queries), results are fed back. After tools return, the model writes the final analyst response.

Anti-hallucination: System prompt explicitly states: "Never answer from LLM knowledge alone. If you cannot find the data in the tools, say so explicitly."

Streaming: Phase labels ({ "phase": "Searching athletes..." }) are emitted as SSE events before tool execution to give the user live progress feedback.

## Dossier Summary (`summary.ts`)

•  
M



o  
d  
el:  
g  
pt  
-5  
.6  
-lu  
n  
a  
(a  
n i  
nt  
er  
n  
al  
m  
o  
d  
el  
ali  
as  
) v  
ia  
O  
p  
e  
n  
Al  
I  
nt  
e  
gr  
ati  
o  
n  
•  
Tr  
a  
ns  
p  
or  
t:  
S  
S  
E  
str  
e  
a  
mi  
n  
g  
•  
C  
ac  
h  
e  
d:  
Ye  
s  
—  
st  
or  
e  
d  
in  
at  
hl  
et

es  
.ai  
\_s  
u  
m  
m  
ar  
y;  
re  
-u  
se  
d  
if  
pr  
es  
e  
nt  
u  
nti  
l  
m  
a  
nu  
all  
y r  
e  
g  
e  
n  
er  
at  
e  
d  
•  
Pr  
o  
m  
pt  
:  
St  
ru  
ct  
ur  
e  
d  
m  
ar  
kd  
o  
w  
n  
re  
p  
or  
t c  
ov  
er  
in  
g  
ca  
re  
er  
,  
m  
e  
di  
a  
n  
ar

ra  
tiv  
e,  
co  
nt  
ac  
t  
n  
et  
w  
or  
k,  
int  
ell  
ig  
e  
nc  
e  
as  
se  
ss  
m  
e  
nt

---

## Social Media Backfill (`admin.ts`, `athletes.ts`)

- B  
ul  
k  
b  
ac  
kfi  
ll:  
P  
er  
pl  
ex  
ity  
s  
o  
n  
ar  
-p  
ro  
fo  
r r  
es  
e  
ar  
ch  
!'  
G  
P  
T-  
4  
o-  
mi  
ni  
fo  
r  
ex  
tr  
ac  
tio  
n,  
3

at  
hl  
et  
es

in  
p  
ar  
all  
el

•  
Si  
n  
gl  
e  
at  
hl  
et  
e  
re  
fr  
es  
h:  
S  
a  
m  
e  
tw  
o-  
st  
a  
g  
e  
pi  
p  
eli  
n  
e,  
si  
n  
gl  
e  
at  
hl  
et

1. Wikipedia exact title API (fastest, most accurate)
2. Wikipedia search API with name-token matching guard
3. Perplexity sonar !' URL extraction via GPT-4o-mini
4. Return null if all fail

## Photo Lookup (`photo-lookup.ts`)

### AI Provider Configuration

| Provider                | Package                                  | Base URL                                   | Auth                       |
|-------------------------|------------------------------------------|--------------------------------------------|----------------------------|
| OpenAI                  | @workspace/integrations-openai-ai-server | http://localhost:1106/modelfarm/openai     | Proxy (no user key needed) |
| OpenRouter (Perplexity) | @workspace/integrations-openrouter-ai    | http://localhost:1106/modelfarm/openrouter | Proxy (no user key needed) |

The environment secrets `AI_INTEGRATIONS_OPENAI_API_KEY` and `AI_INTEGRATIONS_OPENROUTER_API_KEY` are set to `_DUMMY_API_KEY_` — actual authentication is handled by the Replit proxy at localhost:1106.

## 8. Authentication & Security

---

### Authentication Provider

Clerk (Replit-managed tenant)

- Frontend: @clerk/react v6
- Backend: @clerk/express v2
- Publishable key resolved at runtime via publishable key Fr

omHost()to support Clerk custom domain

## Token Flow

Because the app runs behind Replit's path-based proxy, session cookies are not reliably forwarded. All API authentication uses Bearer tokens:

- After sign-in, ClerkAuthSync component (in App.tsx) calls `setAuthTokenGetter(getToken)` and `registerTokenGetter(getToken)` — registers the Clerk `getToken` function globally
- All React Query hooks (from `@workspace/api-client-react`) call `setAuthTokenGetter` before each request
- All manual `fetch()` calls in admin components use `authFetch` — a closure wrapping `useAuth().getToken()`
- The API server's `requireAuth` middleware calls `getAuth(req)` from `@clerk/express` to extract `userId` from the Bearer token

## `requireAuth` Middleware`

Located at `artifacts/api-server/src/middleware/requireAuth.ts`.

•  
C  
all  
s  
g  
et  
A  
ut  
h(  
re  
q)  
(  
n  
ot  
r  
e  
q.  
a  
ut  
h  
dir  
ec  
tly  
  
—  
t  
hi  
s  
ca  
us  
e

d  
bu  
gs  
w  
ith  
B  
e  
ar  
er  
t  
ok  
e  
ns  
)

•  
If  
us  
er  
ld

is  
fal  
sy  
!  
4  
0  
1  
{  
er  
ro  
r":  
"

U  
n  
a  
ut  
h  
or  
iz  
e  
d"  
}  
•

S  
et  
s r  
e  
q.  
us  
er  
ld  
fo  
r  
d  
o  
w  
ns  
tr  
e  
a  
m  
ro  
ut  
e  
h  
a  
n  
dl  
er

s

## ``requireAdmin`` Middleware

Defined inline in admin.ts.

- C  
all  
s  
g  
et  
A  
ut  
h(  
re  
q)  
!' r  
es  
ol  
ve  
s  
us  
er  
Id  
!' c  
all  
s  
Cl  
er  
k  
S  
D  
K  
to  
g  
et  
u  
se  
r  
e  
m  
ail
- C  
h  
ec  
ks  
e  
m  
ail  
a  
g  
ai  
ns  
t  
F  
O  
U  
N  
D  
E  
R  
=  
E  
M  
A  
I  
L  
=  
"a  
ni



```
g
e
m
mi
ll
@
th
e
o
ut
si
d
ei
n.
nz
"
•
If
n
ot
fo
u
n
d
er
!'
4
0
3
{"
er
ro
r":
"
F
or
bi
d
d
e
n"
}
```

## Clerk Proxy

A clerkProxyMiddleware (from artifacts/api-server/src/middlewares/clerkProxyMiddleware.ts) proxies Clerk frontend API calls through the server — required for Replit's path-based routing where the frontend and API share the same origin.

## Security Headers (Helmet)

Applied globally via helmet():

```
•
C
S
P:
R
es
tri
ct
s
sc
ri
pt
s t
o
se
lf
+
Cl
```

er  
k  
d  
o  
m  
ai  
ns  
; n  
o  
ex  
te  
rn  
al  
fr  
a  
m  
e  
e  
m  
b  
e  
ds

•  
C  
O  
R  
S:  
O  
nl  
y  
all  
o  
w  
s.  
re  
pli  
t.  
a  
p  
p,  
.r  
e  
pli  
t.  
d  
ev  
,  
a  
n  
d l  
oc  
al  
h  
os  
t  
or  
igi  
ns

•  
B  
o  
dy  
li  
mi  
t:  
1  
M  
B  
m

ax  
r  
e  
q  
u  
es  
t  
b  
o  
dy

•  
R  
e  
q  
u  
es  
t ti  
m  
e  
o  
ut  
: 3  
0-  
se  
co  
n  
d  
gl  
o  
b  
al  
ti  
m  
e  
o  
ut  
vi  
a  
re  
s.  
se  
tTi  
m  
e  
o  
ut  
(3  
0

—  
0  
0  
0)

—  
in  
di  
vi  
d  
u  
al  
ro  
ut  
es  
t  
h  
at  
le  
git  
im

at  
el  
y t  
ak  
el  
o  
n  
g  
er  
c  
all  
r  
es  
.s  
et  
Ti  
m  
e  
o  
ut  
(0  
)  
at  
t  
h  
e  
st  
ar  
t (  
e.  
g.  
b  
ac  
kfi  
ll-  
so  
ci  
al)  
•  
R  
at  
el  
im  
iti  
n  
g:  
C  
o  
nt  
ac  
t f  
or  
m  
(1  
0/  
hr  
),  
C  
h  
ec  
ko  
ut  
(  
1  
0/  
hr  
),  
C  
h

at  
(  
2  
0/  
1  
5  
mi  
n)

## Environment Variables Required

| Variable                            | Used by                | Description                                  |            |
|-------------------------------------|------------------------|----------------------------------------------|------------|
| DATABASE_URL                        | API server, DB package | PostgreSQL connection string                 |            |
| CLERK_SECRET_KEY                    | API server             | Clerk server-side SDK                        |            |
| CLERK_PUBLISHABLE_KEY               | API server             | Clerk publishable key                        |            |
| VITE_CLERK_PUBLISHABLE_KEY          | Frontend               | Clerk publishable key for browser            |            |
| STRIPE_SECRET_KEY                   | API server             | Stripe API                                   |            |
| STRIPE_WEBHOOK_SECRET               | API server             | Stripe webhook signature verification        |            |
| SESSION_SECRET                      | API server             | Express session secret                       |            |
| AI_INTEGRATIONS_OPENAI_API_KEY      | API server             | Set to _DUMMY_API_KEY_ (proxy handles auth)  |            |
| AI_INTEGRATIONS_OPENAI_BASE_URL     | API server             | Replit AI proxy URL                          |            |
| AI_INTEGRATIONS_OPENROUTER_API_KEY  | API server             | Set to _DUMMY_API_KEY_ (proxy handles auth)  |            |
| AI_INTEGRATIONS_OPENROUTER_BASE_URL | API server             | Replit AI proxy URL                          |            |
| TWITTER_BEARER_TOKEN                | API server             | Optional — X API v2 for real follower counts |            |
| LOG_LEVEL                           | API server             | Pino log level (default: info)               |            |
| NODE_ENV                            | Both                   | development \                                | production |

## 9. Admin System

Accessible at /admin. Only shown in the sidebar when the signed-in user's primary email matches anigemmill@theoutsidein.nz. All API calls use authFetch (Bearer token) passed as a prop from AdminPage to sub-components.

### Tab: Customers

•  
C  
all  
s  
G

E  
T  
/a  
pi/  
a  
d  
mi  
n/  
cu  
st  
o  
m  
er  
s

•

S  
h  
o  
w  
s  
all  
C  
ler  
k-  
re  
gi  
st  
er  
e  
d  
us  
er  
s  
m  
er  
g  
e  
d  
wi  
th  
t  
h  
eir

S  
tri  
p  
e  
su  
bs  
cri  
pti  
o  
n  
d  
at  
a

•

Di  
sp  
la  
ys  
: n  
a  
m  
e,  
e  
m  
ail  
, s

u  
bs  
cri  
pti  
o  
n  
pl  
a  
n,  
st  
at  
us  
,  
M  
R  
R,  
tr  
ial  
e  
n  
d,  
si  
g  
n-  
u  
p  
d  
at  
e,  
la  
st  
ac  
tiv  
e  
•  
S  
h  
o  
w  
s  
a  
g  
gr  
e  
g  
at  
e  
st  
at  
s:  
to  
tal  
si  
g  
n-  
u  
ps  
,  
ac  
tiv  
e  
su  
bs  
cri  
b  
er  
s,  
tri  
all

in  
g

## Tab: Enquiries

- C  
all  
s  
G  
E  
T  
/a  
pi/  
a  
d  
mi  
n/  
e  
n  
q  
uir  
ie  
s
- S  
h  
o  
w  
s  
all  
c  
o  
nt  
ac  
t f  
or  
m  
su  
b  
mi  
ss  
io  
ns  
w  
ith  
st  
at  
us  
b  
a  
d  
g  
es
- C  
a  
n  
m  
ar  
k  
as  
r  
e  
a  
d/  
re  
pli  
e  
d



vi  
a  
P  
U  
T  
/a  
pi/  
a  
d  
mi  
n/  
e  
n  
q  
uir  
ie  
s/  
id  
•  
S  
h  
o  
w  
s:  
su  
b  
mi  
tt  
er  
n  
a  
m  
e,  
or  
g.  
e  
m  
ail  
, r  
ol  
e,  
m  
es  
sa  
g  
e  
ex  
ce  
rp  
t,  
ti  
m  
es  
ta  
m  
p

## Tab: Licences

•  
A  
d  
mi  
n  
ta  
b  
ex  
ist  
s  
in

UI  
; n  
o  
b  
ac  
ke  
n  
d  
fu  
nc  
tio  
n  
ali  
ty  
ye  
t

## Tab: AI Usage

•  
A  
d  
mi  
n  
ta  
b  
ex  
ist  
s  
in  
UI  
; n  
o  
b  
ac  
ke  
n  
d  
fu  
nc  
tio  
n  
ali  
ty  
ye  
t

## Tab: Crawl

Three sub-sections:

Crawl Monitor — placeholder text; no live queue/telemetry connected yet

Backfill athlete photos — runs Wikipedia !' Perplexity photo lookup for all athletes without a photo. Safe to run repeatedly (skips athletes that already have a photo). Reports: N found, N skipped.

Backfill social media stats — runs Perplexity sonar-pro lookup for all athletes in parallel batches of 3. Overwrites existing social handles and follower counts. Shows results table with updated handles.

Repopulate athlete intelligence — lists all athletes with their intelligence item count (red if zero) and last crawl date. Per-athlete "Repopulate" button calls POST /api/admin/repopulate/:id, which wipes all existing intelligence data and re-runs the full pipeline. Polls every 4s and shows "Done — N items" when complete.

## Tab: Health

•  
C  
all  
s  
G  
E  
T  
/a

pi/  
h  
e  
alt  
hz

•

S  
h  
o  
w  
s  
A  
PI  
a  
n  
d  
d  
at  
a  
b  
as  
e  
st  
at  
us

•

D  
B  
a  
n  
d  
St  
ri  
p  
e  
h  
e  
alt  
h  
ar  
e i  
nf  
er  
re  
d  
fr  
o  
m  
A  
PI  
s  
er  
ve  
r s  
ta  
rt  
u  
p l  
o  
gs

## Tab: Flags

•

A  
d  
mi  
n  
ta  
b

exists in UI; no backend functionality

### Hardcoded Founder Email

The founder email `anigemmill@theoutsidein.nz` is hardcoded in 5 places that must be kept in sync:

- `artifacts/web/src/App.tsx` — sidebar admin link visibility
- `artifacts/web/src/pages/AdminPage.tsx` — admin page guard
- `artifacts/web/src/components/layout/Sidebar.tsx` — admin nav item
- `artifacts/api-server/src/routes/admin.ts` — `requireAdmin` middleware
- `artifacts/api-server/src/routes/stripe.ts` — `FOUNDER_EMAIL` constant

*TODO: Move to an environment variable `FOUNDER_EMAIL`.*

## 10. Frontend

Framework: React 19 + Vite 7  
Routing: Wouter v3  
State: TanStack React Query v5 (server state); React local state (UI state)  
Styling: Tailwind CSS v4 + Radix UI + shadcn/ui components  
Auth: @clerk/react v6  
All pages are lazy-loaded via `React.lazy()` + `<Suspense>` with a full-page spinner fallback.

### Pages

#### Public Pages (no auth required)

| Page     | Route     | File             | Description                                              |
|----------|-----------|------------------|----------------------------------------------------------|
| Landing  | /         | LandingPage.tsx  | Marketing homepage                                       |
| Pricing  | /pricing  | PricingPage.tsx  | Subscription tier cards; fetches live prices from Stripe |
| About    | /about    | AboutPage.tsx    | Company/product info                                     |
| Contact  | /contact  | ContactPage.tsx  | Demo/enquiry form !' POST /api/contact                   |
| Security | /security | SecurityPage.tsx | Security practices                                       |
| Terms    | /terms    | TermsPage.tsx    | Terms of service                                         |

[Sign In](#)

/sign-in/\*



|         |            |                   |                        |
|---------|------------|-------------------|------------------------|
| Sign Up | /sign-up/* | Inline in App.tsx | Clerk SignUp component |
|---------|------------|-------------------|------------------------|

App Pages (require auth)

| Page            | Route            | File                   | Lines | Description                                            |
|-----------------|------------------|------------------------|-------|--------------------------------------------------------|
| Dashboard       | /app/dashboard   | Dashboard.tsx          | 338   | Overview stats, priority athletes, recent intelligence |
| Feed            | /app/feed        | FeedPage.tsx           | 225   | Intelligence feed across all athletes                  |
| New Athlete     | /app/new-agent   | NewAgentPage.tsx       | 870   | Athlete creation wizard; auto-populates via AI         |
| Dossier         | /app/dossier/:id | DossierPage.tsx        | 1317  | Full athlete profile with tabbed sections              |
| Compare         | /app/compare     | ComparePage.tsx        | 524   | Side-by-side athlete comparison                        |
| Chat            | /app/chat        | ChatPage.tsx           | 577   | AI chat assistant with SSE streaming                   |
| Schedule        | /app/schedule    | SchedulePage.tsx       | 434   | Competition schedule view                              |
| Alerts          | /app/alerts      | AlertsPage.tsx         | 229   | Alert management UI                                    |
| Settings        | /app/settings    | SettingsPage.tsx       | 481   | User preferences, billing link                         |
| Sources         | /app/sources     | SourcesPage.tsx        | 345   | Documents the AI intelligence sources                  |
| Billing Success | /billing-success | BillingSuccessPage.tsx | 59    | Post-checkout confirmation                             |
| Admin           | /admin           | AdminPage.tsx          | 746   | Founder-only admin panel (7 tabs)                      |

DossierPage Tabs

The largest and most complex page (1,317 lines). Tabs:

- Overview statistics



s,  
rank  
ings

,  
P  
B  
s,  
Al  
s  
u  
m  
m  
ar  
y (  
str  
e  
a  
m  
e  
d)

• I  
nt  
ell  
ig  
e  
nc  
e

—  
fil  
te  
re  
d i  
nt  
ell  
ig  
e  
nc  
e  
fe  
e  
d wi  
th  
c  
at  
e  
g  
or  
y t  
a  
bs

•  
Ti  
m  
eli  
n  
e

—  
c  
hr  
o  
n  
ol  
o  
gi  
ca  
l c

ar  
e  
er  
e  
ve  
nt  
s

•  
C  
o  
m  
p  
eti  
tio  
ns

—  
hi  
st  
or  
ic  
al  
re  
su  
lts

+  
u  
pc  
o  
mi  
n  
g  
sc  
h  
e  
d  
ul  
e

•  
C  
o  
nt  
ac  
ts

—  
m  
a  
n  
a  
g  
e  
m  
e  
nt  
/c  
o  
ac  
hi  
n  
g/  
sp  
o  
ns  
or  
n  
et  
w  
or

k  
•  
S  
oc  
ial  
M  
e  
di  
a  
—  
h  
a  
n  
dl  
es  
, f  
oll  
o  
w  
er  
c  
o  
u  
nt  
s,  
"R  
ef  
re  
sh  
fr  
o  
m  
w  
e  
b"  
b  
ut  
to  
n  
•  
Al  
er  
ts  
—  
p  
er  
-c  
at  
e  
g  
or  
y  
al  
er  
t c  
o  
nfi  
g

---

## Navigation

Sidebar (Sidebar.tsx) — visible on all app pages, hidden on public pages.

•  
To  
p  
se  
cti  
o

n:  
A  
p  
p  
l  
o  
g  
o  
+  
gl  
o  
b  
al  
at  
hl  
et  
e  
se  
ar  
ch

•  
At  
hl  
et  
e  
ro  
st  
er  
:  
S  
cr  
oll  
a  
bl  
e l  
ist  
o  
f  
all  
a  
thl  
et  
es  
w  
ith  
a  
va  
ta  
r  
a  
n  
d  
sp  
or  
t

•  
M  
ai  
n  
n  
av  
:  
D  
as  
h  
b  
o  
ar  
d,  
F

e  
e  
d,  
C  
o  
m  
p  
ar  
e,  
C  
h  
at  
,  
S  
ch  
e  
d  
ul  
e,  
Al  
er  
ts  
•  
B  
ot  
to  
m  
n  
av  
:  
S  
o  
ur  
ce  
s,  
S  
et  
tin  
gs  
,  
A  
d  
mi  
n  
(f  
o  
u  
n  
d  
er  
o  
nl  
y),  
S  
ig  
n  
o  
ut  
•  
U  
se  
r  
f  
o  
ot  
er  
:  
A  
va  
ta

ri  
nit  
ial  
s,  
ful  
ln  
a  
m  
e,  
pl  
a  
nl  
a  
b  
el,  
si  
g  
n-  
o  
ut  
b  
ut  
to  
n

AppLayout (AppLayout.tsx) — wraps all app pages. Contains the sidebar + main content area.

PublicLayout (PublicLayout.tsx) — wraps public pages. Minimal header with nav links.

---

## State Management

- S  
er  
ve  
rs  
ta  
te  
:  
Tan  
n  
St  
ac  
k  
R  
e  
ac  
t  
Q  
u  
er  
y.  
All  
A  
PI  
c  
all  
s  
vi  
a  
g  
e  
n  
er  
at  
e  
d  
h  
o  
ok

s f  
ro  
m  
@  
w  
or  
ks  
p  
ac  
e/  
a  
pi-  
cli  
e  
nt  
-r  
e  
ac  
t  
or  
m  
a  
nu  
al  
fet  
ch  
w  
ith  
a  
ut  
h  
F  
et  
ch  
•  
A  
ut  
h  
to  
ke  
ns  
:  
M  
o  
d  
ul  
e-l  
ev  
el  
si  
n  
gl  
et  
o  
n  
in  
g  
et  
A  
ut  
h  
To  
ke  
n.  
ts  
(r  
e  
gi  
st

er  
To  
ke  
n  
G  
et  
te  
r /  
g  
et  
A  
ut  
h  
To  
ke  
n)  
; u  
se  
d  
by  
R  
e  
ac  
t  
Q  
u  
er  
y  
h  
o  
ok  
s.  
A  
d  
mi  
n  
co  
m  
p  
o  
n  
e  
nt  
s  
us  
e  
us  
e  
C  
all  
b  
ac  
k-  
wr  
a  
p  
p  
e  
d  
a  
ut  
h  
F  
et  
ch  
p  
ro  
p  
to



a  
vo  
id  
h  
o  
ok  
o  
rd  
er  
in  
g i  
ss  
u  
es

•  
UI  
st  
at  
e:  
C  
o  
m  
p  
o  
n  
e  
nt  
-le  
ve  
l u  
se  
St  
at  
e

—  
n  
o  
gl  
o  
b  
al  
st  
or  
e  
(Z  
us  
ta  
n  
d/  
R  
e  
d  
ux  
/C  
o  
nt  
ex  
t)  
us  
e  
d

•  
C  
ac  
h  
e i  
nv  
ali  
d

ation: Clerk Query Client Cache Invalidator component calls queryClient.clear() on Clerk user-handleChange() every 5s

## Key Frontend Libraries

| Library                  | Purpose                                      |
|--------------------------|----------------------------------------------|
| @tanstack/react-query v5 | Server state, caching, request deduplication |
|                          |                                              |

wouter v3

|                    |                                           |
|--------------------|-------------------------------------------|
|                    | Lightweight client-side routing           |
| @clerk/react v6    | Auth components and hooks                 |
| lucide-react       | Icon library                              |
| recharts           | Charts (used in comparison and dashboard) |
| framer-motion      | Animations                                |
| radix-ui/*         | Headless accessible UI primitives         |
| react-helmet-async | Document <head> management                |
| react-hook-form    | Form state management                     |
| date-fns           | Date formatting                           |
| xlsx               | Excel export (planned feature)            |
| sonner             | Toast notifications                       |

## Responsive Behaviour

- Si  
d  
e  
b  
ar  
c  
oll  
a  
ps  
es  
o  
n  
m  
o  
bil  
e  
(i  
m  
pl  
e  
m  
e  
nt  
ati  
o  
n  
va  
ri  
es  
p  
er  
c  
o  
m

p  
o  
n  
e  
nt

—  
n  
ot  
f  
ull  
y  
st  
a  
n  
d  
ar  
di  
se  
d)

•  
Ta  
il  
wi  
n  
d  
br  
e  
ak  
p  
oi  
nt  
s  
us  
e  
d  
th  
ro  
u  
g  
h  
o  
ut  
:s  
m:  
,  
m  
d:  
,  
lg:

•  
N  
o  
d  
e  
di  
ca  
te  
d  
m  
o  
bil  
e  
n  
av  
ig  
ati  
o  
n i

implemented yet

---

## 11. Third-Party Integrations

---

### Clerk (Authentication)

- Provider: Replit-managed Clerk token
- Purpose: User authentication, session management

e  
m  
e  
nt  
•  
Fr  
o  
nt  
e  
n  
d:  
@  
cl  
er  
k/r  
e  
ac  
t  
—  
C  
ler  
k  
Pr  
ov  
id  
er  
,  
us  
e  
A  
ut  
h,  
u  
se  
U  
se  
r,  
Si  
g  
nl  
n,  
S  
ig  
n  
U  
p  
•  
B  
ac  
ke  
n  
d:  
@  
cl  
er  
k/  
ex  
pr  
es  
s  
—  
cl  
er  
k  
Mi  
d  
dl  
e  
w

are(  
,  
get  
Auth(  
req)  
, c  
ler  
k  
Client  
•  
Special  
: p  
ublish  
able  
Key  
From  
Host()  
res  
olve  
st  
he  
correct  
key  
based  
on  
host  
name  
—  
require  
ed  
for



Client domain support

•

Proxy: Client Applications are proxied through the API server at /client-proxy/\* via a clerk Proxy Middleware

are  
•  
Now  
we  
book  
s  
are  
currently  
setting  
up  
for  
Clerk  
events  
(  
user  
.create  
ated,  
etc.  
)

## Stripe (Payments)

•  
Provider  
:  
Replit  
Stripe  
integration  
(Stripe  
e-re

pli  
t-s  
yn  
c)

•

P  
ur  
p  
os  
e:  
S  
u  
bs  
cri  
pti  
o  
n  
bil  
lin  
g  
(S  
ta  
rt  
er  
/  
Pr  
o  
/  
E  
nt  
er  
pr  
is  
e)

•

Fr  
o  
nt  
e  
n  
d:  
N  
o  
St  
ri  
p  
e.j  
s

—

r  
e  
dir  
ec  
ts  
to  
S  
tri  
p  
e-  
h  
os  
te  
d  
C  
h  
ec  
ko  
ut  
a

nd  
d  
C  
us  
to  
m  
er  
P  
or  
tal

•  
B  
ac  
ke  
n  
d:  
str  
ip  
e  
n  
p  
m  
p  
ac  
ka  
g  
e;  
S  
T  
RI  
P  
E

—  
S  
E  
C  
R  
E  
T

—  
K  
E  
Y  
e  
nv  
v  
ar

•  
W  
e  
b  
h  
o  
ok  
:  
P  
O  
S  
T  
/a  
pi/  
str  
ip  
e/  
w  
e  
b  
h  
o

ok  
!  
W  
e  
b  
h  
o  
ok  
H  
a  
n  
dl  
er  
s.  
pr  
oc  
es  
s  
W  
e  
b  
h  
o  
ok  
(  
!  
'st  
ri  
p  
e-  
re  
pli  
t-s  
yn  
c  
p  
ac  
ka  
g  
e  
w  
hi  
ch  
m  
ai  
nt  
ai  
ns  
t  
h  
e  
str  
ip  
e.  
\*  
D  
B  
sc  
h  
e  
m  
a  
•  
K  
ey  
d  
es  
ig  
n:  
St

ri  
p  
e  
d  
at  
a  
(c  
us  
to  
m  
er  
s,  
su  
bs  
cri  
pti  
o  
ns  
,  
pr  
o  
d  
uc  
ts,  
p  
ric  
es  
)  
is  
q  
u  
er  
ie  
d  
fr  
o  
m  
str  
ip  
e.  
\*  
sc  
h  
e  
m  
a  
ta  
bl  
es  
vi  
a  
ra  
w  
S  
Q  
L  
—  
n  
ot  
vi  
a  
St  
ri  
p  
e  
A  
PI  
c  
all

s  
—  
fo  
r  
p  
er  
fo  
r  
m  
a  
nc  
e

- Ti  
er  
s:  
St  
ar  
te  
r,  
Pr  
o,  
E  
nt  
er  
pr  
is  
e

—  
p  
ric  
e  
ID  
s f  
et  
ch  
e  
d l  
iv  
e  
fr  
o  
m  
St  
ri  
p  
e  
D  
B

## OpenAI (AI extraction, chat, summaries)

- Pr  
ov  
id  
er  
:  
R  
e  
pli  
t  
AI  
I  
nt  
e  
gr  
ati  
o  
ns

p  
ro  
xy  
.  
P  
ac  
ka  
g  
e:  
@  
w  
or  
ks  
p  
ac  
e/i  
nt  
e  
gr  
ati  
o  
ns  
-o  
p  
e  
n  
ai-  
ai-  
se  
rv  
er  
.  
M  
o  
d  
el  
s  
us  
e  
d:  
g  
pt  
-4  
o  
(e  
xtr  
ac  
tio  
n,  
c  
h  
at  
),  
g  
pt  
-4  
o-  
mi  
ni  
(s  
oc  
ial  
b  
ac  
kfi  
ll  
ex  
tr  
ac



tion)  
,  
g  
pt  
-5  
.6  
-lu  
n  
a  
(d  
os  
si  
er  
s  
u  
m  
m  
ar  
ie  
s  
—  
in  
te  
rn  
al  
m  
o  
d  
el  
ali  
as  
)  
•  
N  
o  
A  
PI  
k  
ey  
  
in  
a  
p  
pli  
ca  
tio  
n  
co  
d  
e  
—  
p  
ro  
xy  
a  
ut  
h  
e  
nti  
ca  
te  
s  
vi  
a  
AI  
\_I  
N  
T

E  
G  
R  
A  
T  
I  
O  
N  
S  
  
-  
O  
P  
E  
N  
A  
I  
  
-  
A  
P  
I  
  
-  
K  
E  
Y  
=  
  
-  
D  
U  
M  
M  
Y  
  
-  
A  
P  
I  
  
-  
K  
E  
Y  
  
-

**OpenRouter / Perplexity (Live web research)**

•  
Pr  
ov  
id  
er  
:  
R  
e  
pli  
t  
A  
I  
I  
nt  
e  
gr  
ati  
o  
ns  
p  
ro  
xy  
!'  
O  
p  
e  
n  
R  
o  
ut  
er  
!'

P  
er  
pl  
ex  
ity

•

P  
ac  
ka  
g  
e:  
@  
w  
or  
ks  
p  
ac  
e/i  
nt  
e  
gr  
ati  
o  
ns  
-o  
p  
e  
nr  
o  
ut  
er  
-ai

•

M  
o  
d  
el  
s  
us  
e  
d:  
p  
er  
pl  
ex  
ity  
/s  
o  
n  
ar  
(  
a  
ut  
o-  
p  
o  
p  
ul  
at  
e  
re  
se  
ar  
ch  
)  
p  
er  
pl  
ex

ity  
/s  
o  
n  
ar  
-p  
ro  
(s  
oc  
ial  
b  
ac  
kfi  
ll,  
re  
fr  
es  
h-  
so  
ci  
al  
—  
hi  
g  
h  
er  
q  
u  
ali  
ty,  
sl  
o  
w  
er  
)  
•  
N  
o  
A  
PI  
k  
ey  
  
in  
a  
p  
pli  
ca  
tio  
n  
co  
d  
e  
—  
p  
ro  
xy  
a  
ut  
h  
e  
nti  
ca  
te  
s  
vi  
a  
AI  
\_I

N  
T  
E  
G  
R  
A  
T  
I  
O  
N  
S  
-  
O  
P  
E  
N  
R  
O  
U  
T  
E  
R  
-  
A  
P  
I  
-  
K  
E  
Y  
=  
-  
D  
U  
M  
M  
Y  
-  
A  
P  
I  
-  
K  
E  
Y  
-

## Wikipedia API

- P  
ur  
p  
os  
e:  
Pr  
ofi  
le  
p  
h  
ot  
o l  
o  
ok  
u  
p
- Di  
re  
ct  
R  
E  
S

T  
A  
PI

—  
n  
o  
S  
D  
K,  
n  
o  
ke  
y r  
e  
q  
uir  
e  
d

•  
U  
se  
r-  
A  
g  
e  
nt  
:  
At  
hl  
et  
el  
nt  
ell  
ig  
e  
nc  
e/  
1.  
0  
(s  
p  
or  
ts-  
int  
ell  
ig  
e  
nc  
e-  
pl  
at  
fo  
r  
m  
)

•  
E  
n  
d  
p  
oi  
nt  
s  
us  
e  
d:  
ac  
tio

n  
=  
q  
u  
er  
y  
&  
pr  
o  
p  
=  
p  
a  
g  
e  
i  
m  
a  
g  
es  
(t  
h  
u  
m  
b  
n  
ail  
),  
ac  
tio  
n  
=  
q  
u  
er  
y  
&li  
st  
=s  
e  
ar  
ch  
(  
n  
a  
m  
e  
se  
ar  
ch  
)

## X (Twitter) API v2

- P  
ur  
p  
os  
e:  
R  
e  
al-  
ti  
m  
e  
T  
wi  
tt  
er  
fo

lower counts

• Auth: Bearertoken via TWITTER

BEARER

TOKEN environment variable

• Option:

If TWITTER

BEARER

TO



K  
E  
N  
is  
n  
ot  
s  
et  
, t  
h  
e  
sy  
st  
e  
m  
fal  
ls  
b  
ac  
k t  
o  
P  
er  
pl  
ex  
ity  
-e  
sti  
m  
at  
e  
d  
co  
u  
nt  
s  
•  
E  
n  
d  
p  
oi  
nt  
:  
G  
E  
T  
/2  
/u  
se  
rs/  
by  
/u  
se  
rn  
a  
m  
e/  
:h  
a  
n  
dl  
e  
?  
us  
er  
.fi  
el  
ds

=  
p  
u  
b  
l  
i  
c\_  
m  
e  
t  
r  
i  
c  
s

## No Email Integration

There is currently no email delivery integration. Transactional email (welcome, alert delivery, billing receipts) is not implemented. Stripe handles billing emails automatically.

## No Analytics Integration

No product analytics, error monitoring (Sentry), or session replay tool is integrated.

# 12. Deployment

## Platform

Replit (cloud workspace) with path-based routing and managed workflows.  
pnpm --filter @workspace/api-server run dev  
pnpm --filter @workspace/web run dev  
pnpm --filter @workspace/mockup-sandbox run dev  
pnpm run build && pnpm run start  
# 1/ node ./build/index.js (bundles ~4MB)  
# 2/ Replit artifact system (index.html is indexed by Replit artifact system)

Frontend:

## Workflows (Replit)

| Workflow                                           | Command                                         | Port                          |
|----------------------------------------------------|-------------------------------------------------|-------------------------------|
| artifacts/api-server: API Server                   | pnpm --filter @workspace/api-server run dev     | 8080                          |
| artifacts/web: web                                 | pnpm --filter @workspace/web run dev            | Assigned by Replit via \$PORT |
| artifacts/mockup-sandbox: Component Preview Server | pnpm --filter @workspace/mockup-sandbox run dev | Assigned by Replit            |

## Production Deployment

Deploy via Replit's "Publish" feature. Replit handles:

•  
S  
S  
L  
te  
r  
mi  
n  
ati  
o  
n  
•  
P  
at  
h-  
b  
as  
e  
d  
ro  
uti  
n  
g  
(

! fr  
o  
nt  
e  
n  
d,  
/  
a  
pi  
! A  
PI  
s  
er  
ve  
r)  
•  
E  
nv  
iro  
n  
m  
e  
nt  
v  
ar  
ia  
bl  
e i  
nj  
ec  
tio  
n  
(s  
a  
m  
e  
se  
cr  
et  
s,  
dif  
fe  
re  
nt  
v  
al  
u  
es  
if  
n  
e  
e  
d  
e  
d)  
•  
H  
e  
alt  
h  
m  
o  
nit  
or  
in  
g

## Required Services for Production

PostgreSQL database — provisioned via Replit database integration

Clerk — Replit-managed tenant; configure custom domain in Clerk dashboard if using a custom domain  
Stripe — live mode keys; configure webhook endpoint to `https://<prod-domain>/api/stripe/webhook`  
Replit AI Integrations — OpenAI and OpenRouter enabled in the Replit workspace

## Database Migrations

```
npm run --filter @workspace/db run db:push # push schema to connected DB  
npm run --filter @workspace/db run db:studio # open Drizzle Studio UI
```

Managed by Drizzle ORM. To apply schema changes:

The stripe- schema is managed automatically by stripe-replit-sync via webhooks — do not hand-edit it.

## 13. Known Issues & Technical Debt

### Bugs

Backfill Photos button overwrites manually corrected photos. If an admin manually patches `avatar_url` for an athlete and then clicks "Backfill Photos", the manual value is overwritten. The name-matching guard in `photo-lookup.ts` reduces (but doesn't eliminate) wrong-person matches. Fix needed: Add an `avatar_url_locked` boolean flag to athletes table; skip locked athletes in backfill.

Social follower counts not refreshed automatically. Counts are only updated via manual admin backfill or the "Refresh from web" button on the dossier. There is no scheduled refresh.

`gpt-5.6-luna` model alias used in dossier summary. This is an undocumented internal model alias in `summary.ts`. If the model alias changes or is deprecated, summaries will fail. Fix: Use `gpt-4o` explicitly.

Alert delivery not implemented. `alert_configs` stores preferences but no delivery mechanism (email, push, webhook) exists.

No real-time Clerk webhook. When a new user signs up, no action is taken server-side (e.g. welcome email, Stripe customer creation). Stripe customer is only created at checkout time.

### Technical Debt

Founder email hardcoded in 5 places. Must be kept in sync manually. Should be extracted to `FOUNDER_EMAIL` environment variable.

No per-user data isolation. All users see the same athlete roster. The platform implicitly assumes a single-agency deployment. Multi-tenant support would require a `user_id` or `org_id` FK on athletes.

`intelligence_count` is a cached denormalized counter. It is manually kept in sync in multiple places (auto-populate, admin repopulate). A DB trigger or a `COUNT(*)` subquery would be more reliable.

No test suite. Zero automated tests — unit, integration, or E2E.

No error monitoring. Errors are logged to Pino stdout but not surfaced to any alerting system (Sentry, etc.).

No rate limiting on most endpoints. Only `/api/contact`, `/api/stripe/checkout`, and `/api/chat` are rate-limited. Admin repopulate could be abused.

`@workspace/api-client-react` generated hooks use a `setAuthTokenGetter` pattern that requires `ClerkAuthSync` to run before any hook fires. Timing edge cases could cause unauthenticated API calls.

Vite proxy not configured for dev API calls. API calls go to the Replit path-based proxy, not a local proxy. This means development requires the API server to be running simultaneously.

`conversations` and `messages` tables exist but the chat page does not persist conversations. Chat history is held in component state only.

`BASE_PATH` and `PORT` must be injected by the Replit artifact system. Running the frontend outside of Replit requires these env vars to be set manually.

## 14. Future Roadmap

Priority order (highest first):

Alert delivery — Email via Resend or SendGrid triggered by scheduled DB polling of `alert_configs`. Send when new intelligence items match an athlete's configured categories.

Continuous crawl background job — Scheduled task (cron or Replit scheduled deployment) that re-runs `autoPopulateAthlete` for every athlete on a configurable interval (default: weekly).

Fix photo backfill overwrite bug — Add `avatar_url_locked` column; skip locked athletes in bulk backfill.

Multi-tenant isolation — Add `org_id` to athletes; enforce RLS or query-level filtering so each agency sees only their roster.

Clerk webhook integration — On `user.created`, create a Stripe customer, send welcome email, optionally create trial subscription.

% - Portal % % % % % % % % % % % % % % % % % %  
% - Webhooks % % % % % % % % % % % % % % % % % %  
% % % % % % % % % % % % % % % % % % % % % % %  
% % % % % % % % % % % % % % % % % % % % % % %  
% stripe-replit-sync  
% (webhook handler)  
%  
% % % % % % % % % % % % % % % % % % % % % % %  
% External APIs %  
% - Wikipedia REST %  
Founder email to env Var Extract FOUNDER\_EMAIL to a Replit secret.  
Test suite — Playwright E2E tests for critical flows (add athlete, view dossier, chat, checkout).  
Error monitoring — Integrate Sentry for error alerting and performance monitoring.  
Request flow (authenticated page load):  
Usage metering — Track AI API calls per user; enforce per-plan limits; surface usage in admin AI Usage tab.  
Browser ! Replit Proxy ! Vite dev server (serves React app)  
Real-time crawl monitor — Emit telemetry from auto-populate jobs; display live queue depth and latency in  
React app ! Clerk (resolves session token)  
admin Crawl tab  
React Query hook ! Replit Proxy ! API Server (Authorization: Bearer <jwt>)  
AI Server expose Auth ! Dossier data response  
For AI features: API Server ! Replit AI Proxy ! OpenAI/Perplexity ! response streamed back  
Athlete tagging and grouping — Allow users to tag athletes (e.g. "Sprint", "Prospects", "Current Clients") and  
filter the roster by tag.

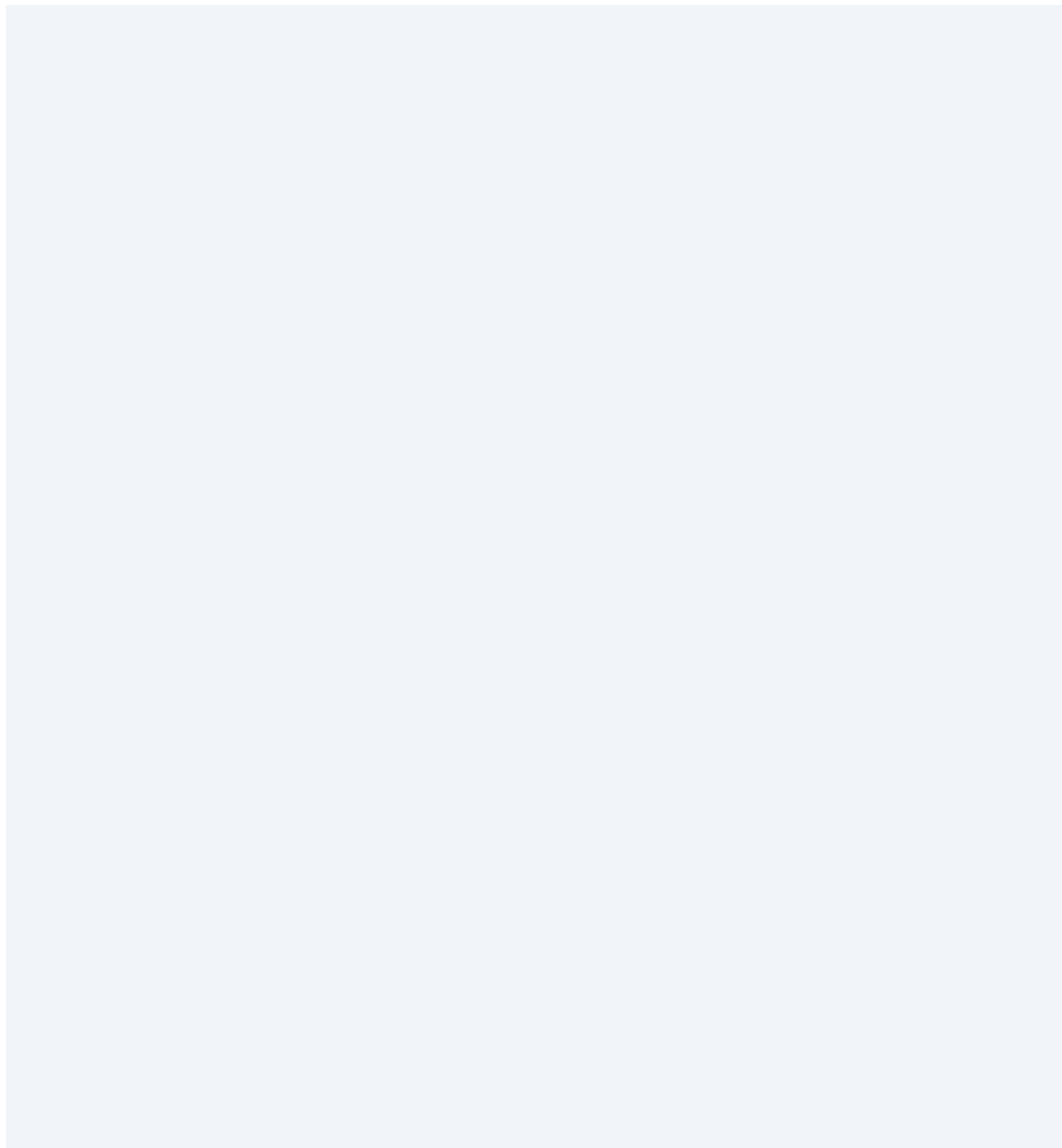
---

## 16. Folder Structure

---

## 15. Architecture Diagram

---



## Why Perplexity before GPT-4o in the pipeline?

GPT-4o's training data has a knowledge cutoff. For a sports intelligence product, current rankings, recent race results, and live social media stats are critical. Perplexity Sonar Pro searches the live web, then GPT-4o extracts structured data from the research text. This two-stage approach gives: (1) real current data from Perplexity, (2) reliable JSON structure from GPT-4o.

## Why store Stripe data in a `stripe.\*` schema?

The stripe-replit-sync integration maintains a live replica of Stripe data in the PostgreSQL database. This avoids Stripe API rate limits, enables JOIN queries against application data, and makes the Pricing page load instantly (no Stripe API call needed).

## Why Drizzle ORM instead of Prisma?

Drizzle was chosen for its lightweight footprint, TypeScript-first schema definition (schema is code, not a separate .prisma file), and excellent performance. The schema-as-code approach also means the @workspace/db package can be imported directly by both the API server and any future services.

## Wikipedia name-match guard

The Wikipedia search fallback (used when an athlete has no exact Wikipedia page) initially caused wrong-person photo assignments (e.g. searching "Brook Macdonald cycling" returned a different athlete's page). A guard was added: search results are only accepted if the result page title contains at least 2 tokens from the athlete's name (first + last name both present). This prevents cross-athlete photo contamination.

## Global 30-second timeout with per-route override

app.ts applies a 30-second response timeout globally to prevent hanging connections. Long-running admin endpoints (e.g. backfill-social which runs Perplexity searches for all athletes) call res.setTimeout(0) at the start of their handler to disable the timeout for that specific request.

## pnpm workspace with version catalog

All shared dependency versions are pinned in pnpm-workspace.yaml under catalog:. This ensures every package in the monorepo uses identical versions of React, Vite, Tailwind, etc. When upgrading a library, update the catalog version once rather than in every package.json.

# 18. README — Running Locally

## Prerequisites

•  
N  
o  
d  
e.js  
2  
0  
+  
•  
p  
n  
p  
m  
9  
+  
•  
A  
P  
os  
tg  
re  
S  
Q  
L  
d  
at  
a  
b  
as  
e  
(lo  
ca

For Replitt-provisioned)

- A Clerk developed permanent account

- A Stripper account (test mode)

- Replitt Agent integration successful (for

```

1) # On the terminal, Set the stripe secret key automatically. The stripe API key is not a secret, but the variables are handled by the
2) # Stripe CLI. Run stripe locally via Stripe CLI. KEY values work as-is.
3) # stripe secret key is forwarded to localhost:8080/api/stripe/webhook
4) # stripe install
5) # stripe open workspace/api-server run dev
6) # stripe listen --secret-key=sk_test_1234567890 --port 8080
7) # stripe open workspace/api-server run dev
8) # stripe open workspace/web run dev
9) # stripe open http://localhost:3000

```

## 5. Start the frontend

Create a Clerk application at <https://clerk.com>  
Set the CLERK\_SECRET\_KEY and CLERK\_PUBLISHABLE\_KEY / VITE\_CLERK\_PUBLISHABLE\_KEY  
from your Clerk dashboard  
In Clerk dashboard !' JWT templates !' ensure the default session token is configured  
If running locally, set VITE\_CLERK\_PROXY\_URL="" (empty string)

```
Copy the webhook signing secret to STRIPE_WEBHOOK_SECRET
curl http://localhost:8080/api/healthz
# {"status":"ok"}
```

Visit <http://localhost:3000> and sign up for an account.

| Issue                                      | Fix                                         |
|--------------------------------------------|---------------------------------------------|
| PORT environment variable is required      | Set PORT=3000 before running the frontend   |
| BASE_PATH environment variable is required | Set BASE_PATH=/ before running the frontend |



DATABASE\_URL must be set

Ensure DATABASE\_URL is in your environment

Admin page shows 403

Your Clerk email doesn't match FOUNDER\_EMAIL in admin.ts

AI features return errors

Check AI\_INTEGRATIONS\_\* env vars point to valid endpoints

Photo backfill returns wrong person

Click "Repopulate" for the athlete instead — uses better search strategy

```
# Open Drizzle Studio (DB browser)
pnpm --filter @workspace/db run db:studio
pnpm --filter @workspace/api-server run typecheck
pnpm --filter @workspace/web run typecheck
pnpm --filter @workspace/db run db:push
```

Running typecheck

Database management